

Missing feature-test macros 2017-2019

Document #: P1902R1
Date: 2019-11-08
Project: Programming Language C++
CWG, LWG
Reply-to: Barry Revzin
<barry.revzin@gmail.com>

Contents

- [1 Revision history](#)
- [2 Introduction](#)
 - [2.1 NB Comments](#)
- [3 `constexpr` policy](#)
- [4 2017](#)
 - [4.1 Toronto \(201707\)](#)
 - [4.2 Albuquerque \(201711\)](#)
- [5 2018](#)
 - [5.1 Jacksonville \(201803\)](#)
 - [5.2 Rapperswil \(201806\)](#)
 - [5.3 San Diego \(201811\)](#)
- [6 2019](#)
 - [6.1 Kona \(201902\)](#)
 - [6.2 Cologne \(201907\)](#)
- [7 Wording](#)
- [8 References](#)

§ 1 Revision history

R0 [[P1902R0](#)] of this paper was presented to SG10 in Belfast in November, 2019. Some of the macros were renamed to be more consistent with other design decisions, and a couple more were added for papers that were missed. We also came up with a [constexpr policy](#).

§ 2 Introduction

When [[P0941R2](#)] was adopted both into the standard and as policy, that paper thoroughly went through all the relevant features that existed at the time. At the Cologne meeting in July 2019, we added signs in both the Core and Library Working Group rooms to ensure that papers in flight did not forget - they had to either explicitly adopt a new feature-test macro or explicitly state why they did not need one.

In the intervening time-frame, several meetings' worth of papers were adopted which did not have a feature-test macro. The goal of this paper is to go through all of those papers and add the missing ones.

As a general note, [[SD6](#)] has been updated and is current through Cologne and through a few LWG issues following that.

§ 2.1 NB Comments

This paper resolves the following NB comments:

- [FI015](#)
- [GB146](#): add a macro for concepts.
- [GB147](#): add a macro for `constexpr`.
- [US150](#): add a macro for familiar syntax for generic lambdas.
- [US167](#): add a macro for non-member `ssize` `()`
- [DE168](#): providing a consistent policy for `constexpr`

§ 3 `constexpr` policy

In the previous draft of this paper, there was a discussion on whether or not we want fine-grained or coarse-grained feature test macros for all the `constexpr` extensions.

The issue is: we keep extending both the language and the library functionality for what you can do at constant evaluation time. How do we test for those? If we only have coarse-grained macros, this is hostile to implementers (because it requires a strict, linear adoption of features, some of which may be both large and unrelated) and this is hostile to users (because they check this one huge umbrella macro). If we have overly fine macros, this is harmful to the implementation as it adds more macros that compilers need to synthesize, and just adds more things that we have to check.

The policy discussed and agreed upon by SG10 in Belfast is as follows:

- For the language, we will have a single `__cpp_constexpr` macro. It will be bumped every time we extend `constexpr` in the language.
- For the library, we will add a specific feature test macros for significant, special features. Otherwise, for those cases where we are just adding `constexpr` to more things in the library, we will have a dedicated test macro *per header* and just bump that header-specific macro on each change. That macro will be named `__cpp_lib_constexpr_HEADER` (with the exception of a few preexisting macros for array and algorithm which have slightly different names).

This policy leads to several changes from R0 of this paper.

§ 4 2017

§ 4.1 Toronto (201707)

[\[P0683R1\]](#) (Default member initializers for bit-fields): no macro necessary.

[\[P0704R1\]](#) (Fixing const-qualified pointers to members): no macro necessary.

[\[P0409R2\]](#) (Allow lambda capture `[=, this ]`): no macro necessary.

[\[P0306R4\]](#) (Comma omission and comma deletion): no macro necessary.

[P0329R4] (Designated Initialization Wording): Marc Mutz suggests a use case where you might only provide a constructor if designated initialization is available (using designated initializers as a proxy for named parameters). The following is slightly reduced from his example:

```
class QSlider {
public:
#ifdef __cpp_designated_initializers
    struct Properties {
        struct { int min, max; } range = {};
        bool showTickMarks = false;
    };
    explicit QSlider(Properties props);
#else
    // traditional, pre-existing approach:
    void setShowTickMarks(bool);
    bool showTickMarks() const noexcept;

    void setRange(int min, int max);
    int minimum() const noexcept;
    int maximum() const noexcept;
};
```

Where a C++14 user could then do:

```
#ifdef __cpp_designated_initializers
    auto sl = new QSlider({ .range = {0, 100}, .showTickMarks = true}, this);
#else
    auto sl = new QSlider(this);
    sl->setRange(0, 100);
    sl->setShowTickMarks(true);
#endif
```

This paper proposes the feature test macro `__cpp_designated_initializers`.

[P0428R2] (Familiar template syntax for generic lambdas): this paper proposes to bump the macro `__cpp_generic_lambdas`. One of the things this feature allows for is, for instance, defaulting a template parameter on a lambda, which is arguably a feature enhancement:

```
struct X { int i, j; };

auto f =
#ifdef __cpp_generic_lambdas >= 201707
    [<class T=X>(T&& var)
#else
    [&](auto&& var)
#endif
    {
        };

// having the macro allows this usage
f({1, 2});
```

[P0702R1] (Language support for Constructor Template Argument Deduction): no macro necessary.

[P0734R0] (Wording Paper, C++ extensions for Concepts): this paper proposes the macro `__cpp_concepts`, which will be bumped later, but for SD-6 purposes will start here with the value 201707.

[P0463R1] (Endian just Endian) to the C++ working paper: Already have a macro from [P1612R1].

[P0682R1] (Repairing elementary string conversions): Already have a macro from [LWG3137].

[P0674R1] (Extending make_shared to Support Arrays): [this paper proposes to bump __cpp_lib_shared_ptr_arrays](#), since make_shared can save an allocation.

§ 4.2 Albuquerque (201711)

[P0614R1] (Range-based for statements with initializer): no macro necessary.

[P0588R1] (Simplifying implicit lambda capture): no macro necessary.

[P0846R0] (ADL and Function Templates that are not Visible): no macro necessary.

[P0641R2] (Resolving Core Issue #1331 (const mismatch with defaulted copy constructor)): no macro necessary.

[P0859R0] (Core Issue 1581: When are constexpr member functions defined?): [this paper proposes the macro __cpp_constexpr_in_decltype](#). This issue is a blocker for library being able to implement [P1065R2], so the library needs to know when to be able to do that. It's unclear if users outside of standard library implementers will need this.

[P0515R3] (Consistent comparison) and [P0768R1] (Library Support for the Spaceship (Comparison) Operator): already have a macro.

[P0857R0] (Wording for “functionality gaps in constraints”): no macro necessary.

[P0692R1] (Access Checking on Specializations): no macro necessary.

[P0624R2] (Default constructible and assignable stateless lambdas): no macro necessary.

[P0767R1] (Deprecate POD): no macro necessary.

[P0315R4] (Wording for lambdas in unevaluated contexts): no macro necessary.

[P0550R2] (Transformation Trait remove_cvref): [this paper proposes the macro __cpp_lib_remove_cvref](#).

[P0777R1] (Treating Unnecessary decay): no macro necessary.

[P0600R1] (nodiscard in the Library): no macro necessary.

[P0439R0] (Make std ::memory_order a scoped enumeration): no macro necessary

[P0053R7] (C++ Synchronized Buffered Ostream): [this paper proposes the macro __cpp_lib_syncbuf](#).

[P0653R2] (Utility to convert a pointer to a raw pointer): [this paper proposes the macro __cpp_lib_to_address](#).

[P0202R3] (Add constexpr modifiers to functions in <algorithm> and <utility> Headers): a macro was already added.

[P0415R1] (Constexpr for std ::complex): [this paper proposes to introduce the macro __cpp_lib_constexpr_complex](#). That is, introducing a new macro for this header.

[P0718R2] (Atomic shared_ptr): [this paper proposes the macro __cpp_lib_atomic_shared_ptr](#).

[P0020R6] (Floating Point Atomic): [this paper proposes the macro __cpp_lib_atomic_float](#).

[P0616R0] (de-pessimize legacy `<numeric` `>` algorithms with `std::move`): no macro necessary.

[P0457R2] (String Prefix and Suffix Checking): [this paper proposes the macro `\_\_cpp\_lib\_starts\_ends\_with`](#).

§ 5 2018

§ 5.1 Jacksonville (201803)

[P0840R2] (Language support for empty objects: already has a macro.

[P0962R1] (Relaxing the range-for loop customization point finding rules): no macro necessary.

[P0969R0] (Allow structured bindings to accessible members): no macro necessary.

[P0961R1] (Relaxing the structured bindings customization point finding rules): no macro necessary.

[P0634R3] (Down with `typename!`): no macro necessary.

[P0780R2] (Allow pack expansion in lambda init-capture): [this paper proposes to bump the macro `\_\_cpp\_init\_captures`](#). Having such a macro would allow you to avoid using `tuple` where necessary. The motivating example in that paper could thus conditionally improve compile throughput:

```
template <class... Args>
auto delay_invoke_foo(Args... args) {
    #if __cpp_init_captures >= 201803
        return [...args=std::move(args)]() -> decltype(auto) {
            return foo(args...);
        };
    #else
        return [tup=std::make_tuple(std::move(args)...)]() -> decltype(auto) {
            return std::apply([](auto const&... args) -> decltype(auto) {
                return foo(args...);
            }, tup);
        };
    #endif
}
```

[P0479R5] (Proposed wording for likely and unlikely attributes (Revision 5)): already have a macro.

[P0905R1] (Symmetry for spaceship): in a vacuum, maybe, but since there isn't an implementation of `<=>` that includes just up to this point, it's probably not worth it.

[P0754R2] (`<version` `>`): already checkable with `__has_include`.

[P0809R0] (Comparing Unordered Containers): no macro necessary.

[P0355R7] (Extending chrono to Calendars and Time Zones): [this paper proposes to bump the macro `\_\_cpp\_lib\_chrono`](#).

[P0966R1] (string `::reserve` Should Not Shrink): no macro necessary.

[P0551R3] (Thou Shalt Not Specialize `std` Function Templates!): no macro necessary.

[P0753R2] (Manipulators for C++ Synchronized Buffered Ostream): [this paper proposes to bump the macro `\_\_cpp\_lib\_syncbuf`](#), which is also added by this paper.

[P0122R7] (`<span >`): already has a macro by way of [LWG3274].

[P0858R0] (Constexpr iterator requirements): already has a macro.

§ 5.2 Rapperswil (201806)

[P0806R2] (Deprecate implicit capture of `this` via `[=]`): no macro necessary.

[P1042R1] (`__VA_OPT__` wording clarifications): no macro necessary.

[P0929R2] (Checking for abstract class types): no macro necessary.

[P0732R2] (Class Types in Non-Type Template Parameters): already has a macro.

[P1025R1] (Update The Reference To The Unicode Standard): no macro necessary.

[P0528R3] (The Curious Case of Padding Bits, Featuring Atomic Compare-and-Exchange): no macro necessary.

[P0722R3] (Efficient sized delete for variable sized classes): already has a macro.

[P1064R0] (Allowing Virtual Function Calls in Constant Expressions): [this paper proposes to bump `\_\_cpp\_constexpr`](#). There are functions that include `virtual` calls that would be able to be made `constexpr`. This macro already has a higher value in the standard, so no wording changes necessary - just to track in SD-6.

[P1008R1] (Prohibit aggregates with user-declared constructors): no macro necessary.

[P1120R0] (Consistency improvements for `<=>` and other comparison operators): no macro necessary.

[P0542R5] (Support for contract based programming in C++): would have been checked by the attribute.

[P0941R2] (Integrating feature-test macros into the C++ WD (rev. 2)): very meta.

[P0892R2] (`explicit` (`bool`)): already has a macro.

[P0476R2] (Bit-casting object representations): already has a macro.

[P0788R3] (Standard Library Specification in a Concepts and Contracts World): no macro necessary.

[P0458R2] (Checking for Existence of an Element in Associative Containers): no macro necessary. The new algorithms are nicer to use, but if you need to support the old code, then the old code works just as well.

[P0759R1] (fpos Requirements): no macro necessary.

[P1023R0] (constexpr comparison operators for `std::array`): [this paper proposes to bump the macro `\_\_cpp\_lib\_array\_constexpr`](#).

[P0769R2] (Add shift to `<algorithm >`): [this paper proposes the macro `\_\_cpp\_lib\_shift`](#).

[P0887R1] (The identity metafunction): [this paper proposes the macro `\_\_cpp\_lib\_type\_identity`](#).

[P0879R0] (Constexpr for swap and swap related functions): already has a macro.

[P0758R1] (Implicit conversion traits and utility functions). [this paper proposes the macro `\_\_cpp\_lib\_nothrow\_convertible`](#).

[P0556R3] (Integral power-of-2 operations): [this paper proposes the macro `\_\_cpp\_lib\_int\_pow2`](#). [Editor's note: There are NB comments about renaming the features introduced by this paper, which may necessitate renaming this macro]

[P0019R8] (Atomic Ref): [this paper proposes the macro `\_\_cpp\_lib\_atomic\_ref`](#).

[P0935R0] (Eradicating unnecessarily explicit default constructors from the standard library): no macro necessary.

[P0646R1] (Improving the Return Value of Erase-Like Algorithms): already has a macro.

[P0619R4] (Reviewing Deprecated Facilities of C++17 for C++20): no macro necessary.

[P0898R3] (Standard Library Concepts): already has a macro.

§ 5.3 San Diego (201811)

[P0982R1] (Weaken Release Sequences): no macro necessary.

[P1084R2] (Today's `_return-type-requirement_s` Are Insufficient): [this paper proposes to bump `\_\_cpp\_concepts`](#), which was also added by this paper.

[P1131R2] (Core Issue 2292: *simple-template-id* is ambiguous between *class-name* and *type-name*): no macro necessary.

[P1289R1] (Access control in contract conditions): no macro necessary

[P1002R1] (Try-catch blocks in constexpr functions): [this paper proposes to bump `\_\_cpp\_constexpr`](#), as per previous arguments to allow more functions to be declared `constexpr`. This one more important than the rest as actually writing `try` in a function already makes `constexpr` ill-formed.

[P1327R1] (Allowing `dynamic_cast`, polymorphic typeid in Constant Expressions): [this paper proposes to bump `\_\_cpp\_constexpr`](#).

[P1236R1] (Alternative Wording for P0907R4 Signed Integers are Two's Complement): no macro necessary.

[P0482R6] (`char8_t`: A type for UTF-8 characters and strings (Revision 6)): already has a macro.

[P1353R0] (Missing Feature Test Macros): already adopted.

[P1073R3] (Immediate functions): [this paper proposes the macro `\_\_cpp\_consteval`](#). There are some functions that you really only want to run at compile time. You cannot enforce this in C++17, but having this macro would allow you to conditionally enforce it as it becomes available.

[P0595R2] (`std::is_constant_evaluated`): already has a macro.

[P1141R2] (Yet another approach for constrained declarations): no macro necessary.

[P1094R2] (Nested Inline Namespaces): no macro necessary.

[P1330R0] (Changing the active member of a union inside constexpr): [this paper proposes to bump __cpp_constexpr](#), as per previous arguments. This would allow making `std::optional` fully `constexpr`, for instance.

[P1123R0] (Editorial Guidance for merging P0019r8 and P0528r3): no macro necessary.

[P0487R1] (Fixing `operator >>(basic_istream &, CharT*)` (LWG 2499)): no macro necessary.

[P0602R4] (variant and optional should propagate copy/move triviality): no macro necessary.

[P0655R1] (`visit <R>`: Explicit Return Type for visit): no macro necessary.

[P0972R0] (`<chrono> zero()`, `min()`, and `max()` should be `noexcept`): no macro necessary.

[P1006R1] (Constexpr in `std::pointer_traits`): [this paper proposes to bump the macro __cpp_lib_constexpr_memory](#).

[P1032R1] (Misc constexpr bits): This paper originally added a single feature test macro, `__cpp_lib_constexpr_misc`, that was since renamed by [P1424R1] to `__cpp_lib_constexpr`. Per the policy laid out in this paper, [this paper proposes to remove __cpp_lib_constexpr and instead add separate macros for each header: __cpp_lib_constexpr_functional, __cpp_lib_constexpr_iterator, __cpp_lib_constexpr_string_view, __cpp_lib_constexpr_tuple, and __cpp_lib_constexpr_utility and to bump __cpp_lib_array_constexpr](#).

[P1148R0] (Cleaning up Clause 20): no macro necessary.

[P0318R1] (`unwrap_ref_decay` and `unwrap_reference`): [this paper proposes the macro __cpp_lib_unwrap_ref](#).

[P0357R3] (`reference_wrapper` for incomplete types): no macro necessary.

[P0608R3] (A sane variant converting constructor): no macro necessary.

[P0771R1] (`std::function` move constructor should be `noexcept`): no macro necessary.

[P1007R3] (`std::assume_aligned`): [this paper proposes the macro __cpp_lib_assume_aligned](#).

[P1020R1] (Smart pointer creation with default initialization): [this paper proposes the macro __cpp_lib_smart_ptr_default_init](#).

[P1285R0] (Improving Completeness Requirements for Type Traits): no macro necessary.

[P1248R1] (Remove `CommonReference` requirement from `StrictWeakOrdering` (a.k.a `Fixing Relations`): no macro necessary.

[P0591R4] (Utility functions to implement `uses_allocator` construction): no macro necessary.

[P0899R1] (LWG 3016 is Not a Defect): no macro necessary.

[P1085R2] (Should `Span` be `Regular`?): no macro necessary.

[P1165R1] (Make stateful allocator propagation more consistent for `operator +` (`basic_string`)): no macro necessary.

[P0896R4] (The One Ranges Proposal): already has a macro.

[P0356R5] (Simplified partial function application): already has a macro.

[P0919R3] (Heterogeneous lookup for unordered containers): already has a macro.

[P1209R0] (Adopt Consistent Container Erasure from Library Fundamentals 2 for C++20): already has a macro.

§ 6 2019

§ 6.1 Kona (201902)

[P1286R2] (Contra CWG DR1778): no macro necessary.

[P1091R3] (Extending structured bindings to be more like variable declarations): no macro necessary.

[P1381R1] (Reference capture of structured bindings): no macro necessary.

[P1041R4] (Make `char16_t/` `char32_t` string literals be UTF-16/32): no macro necessary.

[P1139R2] (Address wording issues related to ISO 10646): no macro necessary.

[P1323R2] (Contract postconditions and return type deduction): contracts were removed.

[P0960R3] (Allow initializing aggregates from a parenthesized list of values): already has a macro.

[P1009R2] (Array size deduction in new-expressions): no macro necessary.

[P1103R3] (Merging Modules): already has a macro

[P1185R2] (`<=>` `!=` `==`): this paper proposes to bump `_cpp_impl_three_way_comparison`.

[P0339R6] (`polymorphic_allocator` `<>` as a vocabulary type): this paper proposes the macro `_cpp_lib_polymorphic_allocator`.

[P0340R3] (Making `std` `::underlying_type` SFINAE-friendly): no macro necessary.

[P0738R2] (I Stream, You Stream, We All Stream for `istream_iterator`): no macro necessary.

[P1458R1] (Mandating the Standard Library: Clause 16 - Language support library): no macro necessary.

[P1459R1] (Mandating the Standard Library: Clause 18 - Diagnostics library): no macro necessary.

[P1462R1] (Mandating the Standard Library: Clause 20 - Strings library): no macro necessary.

[P1463R1] (Mandating the Standard Library: Clause 21 - Containers library): no macro necessary.

[P1464R1] (Mandating the Standard Library: Clause 22 - Iterators library): no macro necessary.

[P1164R1] (Make `create_directory` `()` Intuitive): no macro necessary.

[P0811R3] (Well-behaved interpolation for numbers and pointers): already has a macro.

[P1001R2] (Target Vectorization Policies from Parallelism V2 TS to C++20): no macro necessary.

[P1227R2] (Signed `ssize` functions, unsigned `size` functions): [this paper proposes the macro `\_\_cpp\_lib\_ssize`](#).

[P1252R2] (Ranges Design Cleanup): no macro necessary.

[P1024R3] (Usability Enhancements for `std::span`): already has a macro.

[P0920R2] (Precalculated hash values in lookup): already had a macro, which was subsequently removed.

[P1357R1] (Traits for [Un]bounded Arrays): no macro necessary

§ 6.2 Cologne (201907)

[P1161R3] (Deprecate uses of the comma operator in subscripting expressions): no macro necessary.

[P1331R2] (Permitting trivial default initialization in `constexpr` contexts): already has a macro.

[P0735R1] (Interaction of `memory_order_consume` with release sequences): already has a macro.

[P0848R3] (Conditionally Trivial Special Member Functions): no macro necessary. Would people really write both?

[P1186R3] (When do you actually use `<=>`?): this paper proposed to bump `__cpp_impl_three_way_comparison`, but shouldn't have. This paper doesn't need a macro.

[P1301R4] (`[[nodiscard ("should have a reason")]]`): already has a macro.

[P1099R5] (Using Enum): already has a macro

[P1630R1] (Spaceship needs a tune-up): [this paper proposes that the bump of `\_\_cpp\_impl\_three\_way\_comparison` is associated with this paper instead of P1186R3](#).

[P1616R1] (Using unconstrained template template parameters with constrained templates): no macro necessary.

[P1816R0] (Wording for class template argument deduction for aggregates): no macro necessary. Would you choose to not provide a deduction guide for an aggregate?

[P1668R1] (Enabling `constexpr` Intrinsics By Permitting Unevaluated inline-assembly in `constexpr` Functions): [this paper proposes to bump `\_\_cpp\_constexpr`](#).

[P1766R1] (Mitigating minor modules maladies): already has a macro.

[P1811R0] (Relaxing redefinition restrictions for re-exportation robustness): already has a macro.

[P0388R4] (Permit conversions to arrays of unknown bound): no macro necessary.

[P1823R0]: no macro necessary.

[P1143R2] (Adding the `constinit` keyword): already has a macro.

[P1452R2] (On the non-uniform semantics of return-type-requirements): [this paper proposes to bump the value of `\_\_cpp\_concepts`](#).

[P1152R4] (Deprecating `volatile`): no macro necessary.

[P1771R1] (`[[nodiscard]]` for constructors): already has a macro.

[P1814R0] (Wording for Class Template Argument Deduction for Alias Templates): already has a macro.

[P1825R0] (Merged wording for P0527R1 and P1155R3): no macro needed, just write `std::move()`.

[P1703R1] (Recognizing Header Unit Imports Requires Full Preprocessing): no macro needed.

[P0784R7] (More constexpr containers): already has a macro. This is a significant library feature that merits its own macro (`__cpp_lib_constexpr_dynamic_alloc`).

[P1355R2] (Exposing a narrow contract for `ceil2`): no macro needed.

[P0553R4] (Bit operations): already has a macro.

[P1424R1] (`constexpr` feature macro concerns): already resolved.

[P0645R10] (Text Formatting): already has a macro.

[P1361R2] (Integration of chrono with text formatting): already has a macro.

[P1652R1] (Printf corner cases in `std::format`): already has a macro.

[P0631R8] (Math Constants): already has a macro.

[P1135R6] (The C++20 Synchronization Library), [P1643R1] (Add wait/notify to `atomic_ref`), and [P1644R0] (Add wait/notify to `atomic<shared_ptr>`): already have macros.

[P1466R3] (Miscellaneous minor fixes for chrono): already has a macro.

[P1754R1] (Rename concepts to `standard_case` for C++20, while we still can): [this paper proposes to bump `\_\_cpp\_lib\_concepts`](#), even though nobody implements this yet, because otherwise it's confusing.

[P1614R2] (The Mothership has Landed): [erroneously introduced the new macro `\_\_cpp\_lib\_spaceship`, this paper proposes removing that macro and instead bumping `\_\_cpp\_lib\_three\_way\_comparison`](#).

[P0325R4] (`to_array` from LFTS with updates): already has a macro.

[P0408R7] (Efficient Access to `basic_stringbuf`'s Buffer): no macro necessary. Trying to move out of the buffer will compile even without this feature, it just won't be a move.

[P1423R3] (`char8_t` backward compatibility remediation): already has a macro.

[P1502R1] (Standard library header units for C++20): no macro necessary.

[P1612R1] (Relocate Endian's Specification): already has a macro.

[P1661R1] (Remove dedicated precalculated hash lookup interface): already removes a macro.

- [P1650R0] (Output std `::chrono` `::days` with ‘d’ suffix): no macro necessary.
- [P1651R0] (bind_front should not unwrap reference_wrapper): no macro necessary.
- [P1065R2] (Constexpr INVOKE): as above, this paper proposes to rename `__cpp_lib_constexpr_invoke` to `__cpp_lib_constexpr_functional`.
- [P1207R4] (Movability of Single-pass Iterators): no macro necessary.
- [P1035R7] (Input Range Adaptors): this paper proposes to bump the macro `__cpp_lib_ranges`.
- [P1638R1] (basic_istream_view `::iterator` should not be copyable): no macro necessary.
- [P1522R1] (Iterator Difference Type and Integer Overflow): no macro necessary.
- [P1004R2] (Making std `::vector` constexpr): already has a macro.
- [P0980R1] (Making std `::string` constexpr): already has a macro.
- [P0660R10] (Stop Token and Joining Thread, Rev 10): already has a macro.
- [P1474R1] (Helpful pointers for ContiguousIterator): no macro necessary.
- [P1523R1] (Views and Size Types): no macro necessary.
- [P0466R5] (Layout-compatibility and Pointer-interconvertibility Traits): already has a macro.
- [P1208R6] (Adopt source_location for C++20): already has a macro.

§ 7 Wording

Modify table 17 in 15.10 [cpp.predefined] with the following added:

Macro Name	Value	
<code>__cpp_generic_lambdas</code>	201304	<code>201707</code> L
<code>__cpp_init_captures</code>	201304	<code>201803</code> L
<code>__cpp_designated_initializers</code>	<code>201707</code>	L
<code>__cpp_concepts</code>	<code>201907</code>	L
<code>__cpp_constexpr_in_decltype</code>	<code>201711</code>	L
<code>__cpp_consteval</code>	<code>201811</code>	L

Modify table 36 in 17.3.1 [support.limits.general] with the following added:

Macro Name	Value	Header(s)
<code>__cpp_lib_shared_ptr_arrays</code>	201611	<code><memory></code>
	<code>201707</code> L	
<code>__cpp_lib_remove_cvref</code>	<code>201711</code> L	<code><type_traits></code>
<code>__cpp_lib_syncbuf</code>	<code>201803</code> L	<code><syncstream></code>
<code>__cpp_lib_to_address</code>	<code>201711</code> L	<code><memory></code>
<code>__cpp_lib_atomic_shared_ptr</code>	<code>201711</code> L	<code><memory></code>

<code>__cpp_lib_atomic_float</code>	201711	L	<code><atomic></code>
<code>__cpp_lib_starts_ends_with</code>	201711	L	<code><string></code> <code><string_view></code>
<code>__cpp_lib_shift</code>	201806	L	<code><algorithm></code>
<code>__cpp_lib_type_identity</code>	201806	L	<code><type_traits></code>
<code>__cpp_lib_nothrow_convertible</code>	201806	L	<code><type_traits></code>
<code>__cpp_lib_int_pow2</code>	201806	L	<code><bit></code>
<code>__cpp_lib_atomic_ref</code>	201806	L	<code><atomic></code>
<code>__cpp_lib_unwrap_ref</code>	201811	L	<code><type_traits></code>
<code>__cpp_lib_assume_aligned</code>	201811	L	<code><memory></code>
<code>__cpp_lib_smart_ptr_default_init</code>	201811	L	<code><memory></code>
<code>__cpp_lib_polymorphic_allocator</code>	201902	L	<code><memory></code>
<code>__cpp_lib_ssize</code>	201902	L	<code><iterator></code>
<code>__cpp_lib_concepts</code>	201806 201907	L L	<code><concepts></code>
<code>__cpp_lib_spaceship</code>	201907	L	<code><compare></code>
<code>__cpp_lib_constexpr</code>	201811	L	any C++ library header ...
<code>__cpp_lib_array_constexpr</code>	201803 201811	L L	<code><array></code>
<code>__cpp_lib_constexpr_invoke</code> <code>__cpp_lib_constexpr_functional</code>	201907	L	<code><functional></code>
<code>__cpp_lib_constexpr_iterator</code>	201811	L	<code><iterator></code>
<code>__cpp_lib_constexpr_memory</code>	201811	L	<code><memory></code>
<code>__cpp_lib_constexpr_string_view</code>	201811	L	<code><string_view></code>
<code>__cpp_lib_constexpr_tuple</code>	201811	L	<code><tuple></code>
<code>__cpp_lib_constexpr_utility</code>	201811	L	<code><utility></code>
<code>__cpp_lib_chrono</code>	201611 201803	L L	<code><chrono></code>
<code>__cpp_lib_three_way_comparison</code>	201711 201907	L L	<code><compare></code>
<code>__cpp_lib_ranges</code>	201811 201907	L L	<code><algorithm></code> <code><functional></code>

§ 8 References

[LWG3137] S. B.Tam. Header for `__cpp_lib_to_chars`.

<https://wg21.link/lwg3137>

[LWG3274] Jonathan Wakely. Missing feature test macro for `<span>`.

<https://wg21.link/lwg3274>

[P0019R8] Daniel Sunderland, H. Carter Edwards, Hans Boehm, Olivier Giroux, Mark Hoemmen, David Hollman, Bryce Adelstein Lelbach, Jens Maurer. 2018. Atomic Ref.

<https://wg21.link/p0019r8>

- [P0020R6] H. Carter Edwards, Hans Boehm, Olivier Giroux, JF Bastien, James Reus. 2017. Floating Point Atomic.
<https://wg21.link/p0020r6>
- [P0053R7] Lawrence Cowl, Peter Sommerlad, Nicolai Josuttis, Pablo Halpern. 2017. C++ Synchronized Buffered Ostream.
<https://wg21.link/p0053r7>
- [P0122R7] Neil MacIntosh, Stephan T. Lavavej. 2018. span: bounds-safe views for sequences of objects.
<https://wg21.link/p0122r7>
- [P0202R3] Antony Polukhin. 2017. Add Constexpr Modifiers to Functions in `<algorithm>` and `<utility>` Headers.
<https://wg21.link/p0202r3>
- [P0306R4] Thomas Köppe. 2017. Comma elision and comma deletion.
<https://wg21.link/p0306r4>
- [P0315R4] Louis Dionne, Hubert Tong. 2017. Wording for lambdas in unevaluated contexts.
<https://wg21.link/p0315r4>
- [P0318R1] Vicente J. Botet Escribá. 2018. `unwrap_ref_decay` and `unwrap_reference`.
<https://wg21.link/p0318r1>
- [P0325R4] Zhihao Yuan. 2019. `to_array` from LFTS with updates.
<https://wg21.link/p0325r4>
- [P0329R4] Tim Shen, Richard Smith. 2017. Designated Initialization Wording.
<https://wg21.link/p0329r4>
- [P0339R6] Pablo Halpern, Dietmar Kühl. 2019. `polymorphic_allocator<>` as a vocabulary type.
<https://wg21.link/p0339r6>
- [P0340R3] Tim Song. 2019. Making `std::underlying_type` SFINAE-friendly.
<https://wg21.link/p0340r3>
- [P0355R7] Howard E. Hinnant, Tomasz Kamiński. 2018. Extending to Calendars and Time Zones.
<https://wg21.link/p0355r7>
- [P0356R5] Tomasz Kamiński. 2018. Simplified partial function application.
<https://wg21.link/p0356r5>
- [P0357R3] Tomasz Kamiński, Stephan T. Lavavej, Alisdair Meredith. 2018. “`reference_wrapper`” for incomplete types.
<https://wg21.link/p0357r3>
- [P0388R4] Robert Haberlach, Richard Smith. 2019. Permit conversions to arrays of unknown bound.
<https://wg21.link/p0388r4>
- [P0408R7] Peter Sommerlad. 2019. Efficient Access to `basic_stringbuf`’s Buffer.
<https://wg21.link/p0408r7>

- [P0409R2] Thomas Koeppel. 2017. Allow lambda capture [=, this].
<https://wg21.link/p0409r2>
- [P0415R1] Antony Polukhin. 2016. Constexpr for std::complex.
<https://wg21.link/p0415r1>
- [P0428R2] Louis Dionne. 2017. Familiar template syntax for generic lambdas.
<https://wg21.link/p0428r2>
- [P0439R0] Jonathan Wakely. 2016. Make memory_order a scoped enumeration.
<https://wg21.link/p0439r0>
- [P0457R2] Mikhail Maltsev. 2017. String Prefix and Suffix Checking.
<https://wg21.link/p0457r2>
- [P0458R2] Mikhail Maltsev. 2018. Checking for Existence of an Element in Associative Containers.
<https://wg21.link/p0458r2>
- [P0463R1] Howard Hinnant. 2017. endian, Just endian.
<https://wg21.link/p0463r1>
- [P0466R5] Lisa Lippincott. 2019. Layout-compatibility and Pointer-interconvertibility Traits.
<https://wg21.link/p0466r5>
- [P0476R2] JF Bastien. 2017. Bit-casting object representations.
<https://wg21.link/p0476r2>
- [P0479R5] Clay Trychta. 2018. Proposed wording for likely and unlikely attributes.
<https://wg21.link/p0479r5>
- [P0482R6] Tom Honermann. 2018. `char8_t`: A type for UTF-8 characters and strings (Revision 6).
<https://wg21.link/p0482r6>
- [P0487R1] Zhihao Yuan. 2018. Fixing operator>>(basic_istream&, CharT*) (LWG 2499).
<https://wg21.link/p0487r1>
- [P0515R3] Herb Sutter, Jens Maurer, Walter E. Brown. 2017. Consistent comparison.
<https://wg21.link/p0515r3>
- [P0528R3] JF Bastien, Michael Spencer. 2018. The Curious Case of Padding Bits, Featuring Atomic Compare-and-Exchange.
<https://wg21.link/p0528r3>
- [P0542R5] J. Daniel Garcia. 2018. Support for contract based programming in C++.
<https://wg21.link/p0542r5>
- [P0550R2] Walter E. Brown. 2017. Transformation Trait remove_cvref.
<https://wg21.link/p0550r2>
- [P0551R3] Walter E. Brown. 2018. Thou Shalt Not Specialize std Function Templates!
<https://wg21.link/p0551r3>

- [P0553R4] Jens Maurer. 2019. Bit operations.
<https://wg21.link/p0553r4>
- [P0556R3] Jens Maurer. 2018. Integral power-of-2 operations.
<https://wg21.link/p0556r3>
- [P0588R1] Richard Smith. 2017. Simplifying implicit lambda capture.
<https://wg21.link/p0588r1>
- [P0591R4] Pablo Halpern. 2018. Utility functions to implement uses-allocator construction.
<https://wg21.link/p0591r4>
- [P0595R2] Richard Smith, Andrew Sutton, Daveed Vandevoorde. 2018.
`std::is_constant_evaluated`.
<https://wg21.link/p0595r2>
- [P0600R1] Nicolai Josuttis. 2017. `[[nodiscard]]` in the Library.
<https://wg21.link/p0600r1>
- [P0602R4] Zhihao Yuan. 2018. variant and optional should propagate copy/move triviality.
<https://wg21.link/p0602r4>
- [P0608R3] Zhihao Yuan. 2018. A sane variant converting constructor.
<https://wg21.link/p0608r3>
- [P0614R1] Thomas Köppe. 2017. Range-based for statements with initializer.
<https://wg21.link/p0614r1>
- [P0616R0] Peter Sommerlad. 2017. de-pessimise legacy algorithms with `std::move`.
<https://wg21.link/p0616r0>
- [P0619R4] Alisdair Meredith, Alisdair Meredith, Tomasz Kamiński. 2018. Reviewing Deprecated Facilities of C++17 for C++20.
<https://wg21.link/p0619r4>
- [P0624R2] Louis Dionne. 2017. Default constructible and assignable stateless lambdas.
<https://wg21.link/p0624r2>
- [P0631R8] Lev Minkovsky, John McFarlane. 2019. Math Constants.
<https://wg21.link/p0631r8>
- [P0634R3] Nina Ranns, Daveed Vandevoorde. 2018. Down with typename!
<https://wg21.link/p0634r3>
- [P0641R2] Daniel Krügler, Botond Ballo. 2017. Resolving Core Issue #1331 (const mismatch with defaulted copy constructor).
<https://wg21.link/p0641r2>
- [P0645R10] Victor Zverovich. 2019. Text Formatting.
<https://wg21.link/p0645r10>
- [P0646R1] Marc Mutz. 2018. Improving the Return Value of Erase-Like Algorithms I: list/forward list.
<https://wg21.link/p0646r1>

- [P0653R2] Glen Joseph Fernandes. 2017. Utility to convert a pointer to a raw pointer.
<https://wg21.link/p0653r2>
- [P0655R1] Michael Park, Agustín Bergé. 2018. visit: Explicit Return Type for visit.
<https://wg21.link/p0655r1>
- [P0660R10] Nicolai Josuttis, Lewis Baker, Billy O’Neal, Herb Sutter, Anthony Williams. 2019. Stop Token and Joining Thread.
<https://wg21.link/p0660r10>
- [P0674R1] Peter Dimov, Glen Fernandes. 2017. Extending make_shared to Support Arrays.
<https://wg21.link/p0674r1>
- [P0682R1] Jens Maurer. 2017. Repairing elementary string conversions.
<https://wg21.link/p0682r1>
- [P0683R1] Jens Maurer. 2017. Default member initializers for bit-fields.
<https://wg21.link/p0683r1>
- [P0692R1] Matt Calabrese. 2017. Access Checking on Specializations.
<https://wg21.link/p0692r1>
- [P0702R1] Mike Spertus, Jason Merrill. 2017. Language support for Constructor Template Argument Deduction.
<https://wg21.link/p0702r1>
- [P0704R1] Barry Revzin. 2017. Fixing const-qualified pointers to members.
<https://wg21.link/p0704r1>
- [P0718R2] Alisdair Meredith. 2017. Revising atomic_shared_ptr for C++20.
<https://wg21.link/p0718r2>
- [P0722R3] Richard Smith, Andrew Hunter. 2018. Efficient sized delete for variable sized classes.
<https://wg21.link/p0722r3>
- [P0732R2] Jeff Snyder, Louis Dionne. 2018. Class Types in Non-Type Template Parameters.
<https://wg21.link/p0732r2>
- [P0734R0] Andrew Sutton. 2017. Wording Paper, C++ extensions for Concepts.
<https://wg21.link/p0734r0>
- [P0735R1] Will Deacon, Jade Alglave. 2019. Interaction of memory_order_consume with release sequences.
<https://wg21.link/p0735r1>
- [P0738R2] Casey Carter. 2019. I Stream, You Stream, We All Stream for istream_iterator.
<https://wg21.link/p0738r2>
- [P0753R2] Peter Sommerlad, Pablo Halpern. 2018. Manipulators for C++ Synchronized Buffered Ostream.
<https://wg21.link/p0753r2>
- [P0754R2] Alan Talbot. 2018. `<version>`.
<https://wg21.link/p0754r2>

- [P0758R1] Daniel Krügler. 2018. Implicit conversion traits and utility functions.
<https://wg21.link/p0758r1>
- [P0759R1] Daniel Krügler. 2018. fpos requirements.
<https://wg21.link/p0759r1>
- [P0767R1] Jens Maurer. 2017. Deprecate POD.
<https://wg21.link/p0767r1>
- [P0768R1] Walter E. Brown. 2017. Library Support for the Spaceship (Comparison) Operator.
<https://wg21.link/p0768r1>
- [P0769R2] Dan Raviv. 2018. Add shift to .
<https://wg21.link/p0769r2>
- [P0771R1] Nevin Liber, Pablo Halpern. 2018. std::function move constructor should be noexcept.
<https://wg21.link/p0771r1>
- [P0777R1] Walter E. Brown. 2017. Treating Unnecessary decay.
<https://wg21.link/p0777r1>
- [P0780R2] Barry Revzin. 2018. Allow pack expansion in lambda init-capture.
<https://wg21.link/p0780r2>
- [P0784R7] Daveed Vandevoorde, Peter Dimov, Louis Dionne, Nina Ranns, Richard Smith, Daveed Vandevoorde. 2019. More constexpr containers.
<https://wg21.link/p0784r7>
- [P0788R3] Walter E. Brown. 2018. Standard Library Specification in a Concepts and Contracts World.
<https://wg21.link/p0788r3>
- [P0806R2] Thomas Köppe. 2018. Deprecate implicit capture of this via [=].
<https://wg21.link/p0806r2>
- [P0809R0] Titus Winters. 2017. Comparing Unordered Containers.
<https://wg21.link/p0809r0>
- [P0811R3] S. Davis Herring. 2019. Well-behaved interpolation for numbers and pointers.
<https://wg21.link/p0811r3>
- [P0840R2] Richard Smith. 2018. Language support for empty objects.
<https://wg21.link/p0840r2>
- [P0846R0] John Spicer. 2017. ADL and Function Templates that are not Visible.
<https://wg21.link/p0846r0>
- [P0848R3] Barry Revzin, Casey Carter. 2019. Conditionally Trivial Special Member Functions.
<https://wg21.link/p0848r3>
- [P0857R0] Thomas Köppe. 2017. Wording for “functionality gaps in constraints”.
<https://wg21.link/p0857r0>

- [P0858R0] Antony Polukhin. 2017. Constexpr iterator requirements.
<https://wg21.link/p0858r0>
- [P0859R0] Richard Smith. 2017. Core Issue 1581: When are constexpr member functions defined?
<https://wg21.link/p0859r0>
- [P0879R0] Antony Polukhin. 2017. Constexpr for swap and swap related functions.
<https://wg21.link/p0879r0>
- [P0887R1] Timur Doumler. 2018. The identity metafunction.
<https://wg21.link/p0887r1>
- [P0892R2] Barry Revzin, Stephan T. Lavavej. 2018. `explicit` (`bool`).
<https://wg21.link/p0892r2>
- [P0896R4] Eric Niebler, Casey Carter, Christopher Di Bella. 2018. The One Ranges Proposal.
<https://wg21.link/p0896r4>
- [P0898R3] Casey Carter, Eric Niebler. 2018. Standard Library Concepts.
<https://wg21.link/p0898r3>
- [P0899R1] Casey Carter. 2018. LWG 3016 is Not a Defect.
<https://wg21.link/p0899r1>
- [P0905R1] Tomasz Kamiński, Herb Sutter, Richard Smith. 2018. Symmetry for spaceship.
<https://wg21.link/p0905r1>
- [P0919R3] Mateusz Pusz. 2018. Heterogeneous lookup for unordered containers.
<https://wg21.link/p0919r3>
- [P0920R2] Mateusz Pusz. 2019. Precalculated hash values in lookup.
<https://wg21.link/p0920r2>
- [P0929R2] Jens Maurer. 2018. Checking for abstract class types.
<https://wg21.link/p0929r2>
- [P0935R0] Tim Song. 2018. Eradicating unnecessarily explicit default constructors from the standard library.
<https://wg21.link/p0935r0>
- [P0941R2] Ville Voutilainen, Jonathan Wakely. 2018. Integrating feature-test macros into the C++ WD.
<https://wg21.link/p0941r2>
- [P0960R3] Ville Voutilainen, Thomas Köppe. 2019. Allow initializing aggregates from a parenthesized list of values.
<https://wg21.link/p0960r3>
- [P0961R1] Ville Voutilainen. 2018. Relaxing the structured bindings customization point finding rules.
<https://wg21.link/p0961r1>
- [P0962R1] Ville Voutilainen. 2018. Relaxing the range-for loop customization point finding rules.
<https://wg21.link/p0962r1>

- [P0966R1] Mark Zeren, Andrew Luo. 2018. `string::reserve` Should Not Shrink.
<https://wg21.link/p0966r1>
- [P0969R0] Timur Doumler. 2018. Allow structured bindings to accessible members.
<https://wg21.link/p0969r0>
- [P0972R0] Billy Robert O’Neal III. 2018. `zero()`, `min()`, and `max()` should be `noexcept`.
<https://wg21.link/p0972r0>
- [P0980R1] Louis Dionne. 2019. Making `std::string` constexpr.
<https://wg21.link/p0980r1>
- [P0982R1] Hans-J. Boehm, Olivier Giroux, Viktor Vafeiades. 2018. Weaken release sequences.
<https://wg21.link/p0982r1>
- [P1001R2] Alisdair Meredith, Pablo Halpern. 2019. Target Vectorization Policies from Parallelism V2 TS to C++20.
<https://wg21.link/p1001r2>
- [P1002R1] Louis Dionne. 2018. Try-catch blocks in constexpr functions.
<https://wg21.link/p1002r1>
- [P1004R2] Louis Dionne. 2019. Making `std::vector` constexpr.
<https://wg21.link/p1004r2>
- [P1006R1] Louis Dionne. 2018. Constexpr in `std::pointer_traits`.
<https://wg21.link/p1006r1>
- [P1007R3] Timur Doumler, Chandler Carruth. 2018. `std::assume_aligned`.
<https://wg21.link/p1007r3>
- [P1008R1] Timur Doumler, Arthur O’Dwyer, Richard Smith, Howard E. Hinnant, Nicolai Josuttis. 2018. Prohibit aggregates with user-declared constructors.
<https://wg21.link/p1008r1>
- [P1009R2] Timur Doumler. 2019. Array size deduction in new-expressions.
<https://wg21.link/p1009r2>
- [P1020R1] Glen Joseph Fernandes, Peter Dimov. 2018. Smart pointer creation with default initialization.
<https://wg21.link/p1020r1>
- [P1023R0] Tristan Brindle. 2018. constexpr comparison operators for `std::array`.
<https://wg21.link/p1023r0>
- [P1024R3] Tristan Brindle. 2019. Usability Enhancements for `std::span`.
<https://wg21.link/p1024r3>
- [P1025R1] Steve Downey, JeanHeyd Meneide, Martinho Fernandes. 2018. Update The Reference To The Unicode Standard.
<https://wg21.link/p1025r1>
- [P1032R1] Antony Polukhin. 2018. Misc constexpr bits.
<https://wg21.link/p1032r1>

- [P1035R7] Christopher Di Bella, Casey Carter, Corentin Jabot. 2019. Input Range Adaptors.
<https://wg21.link/p1035r7>
- [P1041R4] R. Martinho Fernandes. 2019. Make char16_t/char32_t string literals be UTF-16/32.
<https://wg21.link/p1041r4>
- [P1042R1] Hubert S.K. Tong. 2018. **VA_OPT** wording clarifications.
<https://wg21.link/p1042r1>
- [P1064R0] Peter Dimov, Vassil Vassilev. 2018. Allowing Virtual Function Calls in Constant Expressions.
<https://wg21.link/p1064r0>
- [P1065R2] Tomasz Kamiński, Barry Revzin. 2019. constexpr *INVOKE*.
<https://wg21.link/p1065r2>
- [P1073R3] Richard Smith, Andrew Sutton, Daveed Vandevoorde. 2018. Immediate functions.
<https://wg21.link/p1073r3>
- [P1084R2] Walter E. Brown, Casey Carter. 2018. Today's return-type-requirements Are Insufficient.
<https://wg21.link/p1084r2>
- [P1085R2] Tony Van Eerd. 2018. Should Span be Regular?
<https://wg21.link/p1085r2>
- [P1091R3] Nicolas Lesser. 2019. Extending structured bindings to be more like variable declarations.
<https://wg21.link/p1091r3>
- [P1094R2] Alisdair Meredith. 2018. Nested Inline Namespaces.
<https://wg21.link/p1094r2>
- [P1099R5] Gašper Ažman, Jonathan Mueller. 2019. Using Enum.
<https://wg21.link/p1099r5>
- [P1103R3] Richard Smith. 2019. Merging Modules.
<https://wg21.link/p1103r3>
- [P1120R0] Richard Smith. 2018. Consistency improvements for <=> and other comparison operators.
<https://wg21.link/p1120r0>
- [P1123R0] Daniel Sunderland. 2018. Editorial Guidance for merging P0019r8 and P0528r3.
<https://wg21.link/p1123r0>
- [P1131R2] Jens Maurer. 2018. Core Issue 2292: simple-template-id is ambiguous between class-name and type-name.
<https://wg21.link/p1131r2>
- [P1135R6] David Olsen, Olivier Giroux, JF Bastien, Detlef Vollmann, Bryce Lebach. 2019. The C++20 Synchronization Library.
<https://wg21.link/p1135r6>
- [P1139R2] R. Martinho Fernandes. 2019. Address wording issues related to ISO 10646.
<https://wg21.link/p1139r2>

- [P1141R2] Ville Voutilainen, Thomas Köppe, Andrew Sutton, Herb Sutter, Gabriel Dos Reis, Bjarne Stroustrup, Jason Merrill, Hubert Tong, Eric Niebler, Casey Carter, Tom Honermann, Erich Keane, Walter E. Brown, Michael Spertus, Richard Smith. 2018. Yet another approach for constrained declarations.
<https://wg21.link/p1141r2>
- [P1143R2] Eric Fiselier. 2019. Adding the `constexpr` keyword.
<https://wg21.link/p1143r2>
- [P1148R0] Tim Song. 2018. Cleaning up Clause 20.
<https://wg21.link/p1148r0>
- [P1152R4] JF Bastien. 2019. Deprecating volatile.
<https://wg21.link/p1152r4>
- [P1161R3] Corentin Jabot. 2019. Deprecate uses of the comma operator in subscripting expressions.
<https://wg21.link/p1161r3>
- [P1164R1] Nicolai Josuttis. 2019. Make `create_directory()` intuitive.
<https://wg21.link/p1164r1>
- [P1165R1] Tim Song. 2018. Make stateful allocator propagation more consistent for `operator+(basic_string)`.
<https://wg21.link/p1165r1>
- [P1185R2] Barry Revzin. 2019. `<=>` `!=` `==`.
<https://wg21.link/p1185r2>
- [P1186R3] Barry Revzin. 2019. When do you actually use `<=>`?
<https://wg21.link/p1186r3>
- [P1207R4] Corentin Jabot. 2019. Movability of Single-pass Iterators.
<https://wg21.link/p1207r4>
- [P1208R6] Corentin Jabot, Robert Douglas, Daniel Krugler, Peter Sommerlad. 2019. Adopt source location from Library Fundamentals V3 for C++20.
<https://wg21.link/p1208r6>
- [P1209R0] Alisdair Meredith, Stephan T. Lavavej. 2018. Adopt Consistent Container Erasure from Library Fundamentals 2 for C++20.
<https://wg21.link/p1209r0>
- [P1227R2] Jorg Brown. 2019. Signed `ssize()` functions, unsigned `size()` functions.
<https://wg21.link/p1227r2>
- [P1236R1] Jens Maurer. 2018. Alternative Wording for P0907R4 Signed Integers are Two's Complement.
<https://wg21.link/p1236r1>
- [P1248R1] Tomasz Kamiński. 2018. Remove `CommonReference` requirement from `StrictWeakOrdering`.
<https://wg21.link/p1248r1>
- [P1252R2] Casey Carter. 2019. Ranges Design Cleanup.
<https://wg21.link/p1252r2>

- [P1285R0] Walter E. Brown. 2018. Improving Completeness Requirements for Type Traits.
<https://wg21.link/p1285r0>
- [P1286R2] Richard Smith. 2019. Contra CWG DR1778.
<https://wg21.link/p1286r2>
- [P1289R1] J. Daniel Garcia, Ville Voutilainen. 2018. Access control in contract conditions.
<https://wg21.link/p1289r1>
- [P1301R4] JeanHeyd Meneide, Isabella Muerte. 2019.
[[nodiscard] ("should have a reason")].
<https://wg21.link/p1301r4>
- [P1323R2] Hubert S.K. Tong. 2019. Contract postconditions and return type deduction.
<https://wg21.link/p1323r2>
- [P1327R1] Peter Dimov, Vassil Vassilev, Richard Smith. 2018. Allowing dynamic_cast, polymorphic typeid in Constant Expressions.
<https://wg21.link/p1327r1>
- [P1330R0] Louis Dionne, David Vandevoorde. 2018. Changing the active member of a union inside constexpr.
<https://wg21.link/p1330r0>
- [P1331R2] CJ Johnson. 2019. Permitting trivial default initialization in constexpr contexts.
<https://wg21.link/p1331r2>
- [P1353R0] John Spicer. 2017. Missing Feature Test Macros.
<https://wg21.link/p1353r0>
- [P1355R2] Chris Kennelly. 2019. Exposing a narrow contract for ceil2.
<https://wg21.link/p1355r2>
- [P1357R1] Walter E. Brown, Glen J. Fernandes. 2019. Traits for [Un]bounded Arrays.
<https://wg21.link/p1357r1>
- [P1361R2] Victor Zverovich, Daniela Engert, Howard E. Hinnant. 2019. Integration of chrono with text formatting.
<https://wg21.link/p1361r2>
- [P1381R1] Nicolas Lesser. 2019. Reference capture of structured bindings.
<https://wg21.link/p1381r1>
- [P1423R3] Tom Honermann. 2019. char8_t backward compatibility remediation.
<https://wg21.link/p1423r2>
- [P1424R1] Antony Polukhin. 2019. “constexpr” feature macro concerns.
<https://wg21.link/p1424r1>
- [P1452R2] Hubert Tong. 2019. On the non-uniform semantics of return-type-requirements.
<https://wg21.link/p1452r2>
- [P1458R1] Marshall Clow. 2019. Mandating the Standard Library: Clause 16 - Language support library.
<https://wg21.link/p1458r1>

- [P1459R1] Marshall Clow. 2019. Mandating the Standard Library: Clause 18 - Diagnostics library.
<https://wg21.link/p1459r1>
- [P1462R1] Marshall Clow. 2019. Mandating the Standard Library: Clause 20 - Strings library.
<https://wg21.link/p1462r1>
- [P1463R1] Marshall Clow. 2019. Mandating the Standard Library: Clause 21 - Containers library.
<https://wg21.link/p1463r1>
- [P1464R1] Marshall Clow. 2019. Mandating the Standard Library: Clause 22 - Iterators library.
<https://wg21.link/p1464r1>
- [P1466R3] Howard E. Hinnant. 2019. Miscellaneous minor fixes for chrono.
<https://wg21.link/p1466r3>
- [P1474R1] Casey Carter. 2019. Helpful pointers for ContiguousIterator.
<https://wg21.link/p1474r1>
- [P1502R1] Richard Smith. 2019. Standard library header units for C++20.
<https://wg21.link/p1502r1>
- [P1522R1] Eric Niebler. 2019. Iterator Difference Type and Integer Overflow.
<https://wg21.link/p1522r1>
- [P1523R1] Eric Niebler. 2019. Views and Size Types.
<https://wg21.link/p1523r1>
- [P1612R1] Arthur O'Dwyer. 2019. Relocate Endian's Specification.
<https://wg21.link/p1612r1>
- [P1614R2] Barry Revzin. 2019. The Mothership Has Landed: Adding `<=>` to the Library.
<https://wg21.link/p1614r2>
- [P1616R1] Mike Spertus, Roland Bock. 2019. Using unconstrained template template parameters with constrained templates.
<https://wg21.link/p1616r1>
- [P1630R1] Barry Revzin. 2019. Spaceship needs a tune-up.
<https://wg21.link/p1630r1>
- [P1638R1] Corentin Jabot, Christopher Di Bella. 2019. `basic_istream_view`'s iterator should not be copyable.
<https://wg21.link/p1638r1>
- [P1643R1] David Olsen. 2019. Add wait/notify to `atomic_ref`.
<https://wg21.link/p1643r1>
- [P1644R0] David Olsen. 2019. Add wait/notify to `atomic<shared_ptr>`.
<https://wg21.link/p1644r0>
- [P1650R0] Tomasz Kamiński. 2019. Output `std::chrono::days` with "d" suffix.
<https://wg21.link/p1650r0>

- [P1651R0] Tomasz Kamiński. 2019. `bind_front` should not unwrap `reference_wrapper`.
<https://wg21.link/p1651r0>
- [P1652R1] Zhihao Yuan, Victor Zverovich. 2019. Printf corner cases in `std::format`.
<https://wg21.link/p1652r1>
- [P1661R1] Tomasz Kamiński. 2019. Remove dedicated precalculated hash lookup interface.
<https://wg21.link/p1661r1>
- [P1668R1] Erich Keane. 2019. Enabling `constexpr` Intrinsics By Permitting Unevaluated inline-assembly in `constexpr` Functions.
<https://wg21.link/p1668r1>
- [P1703R1] Boris Kolpackov. 2019. Recognizing Header Unit Imports Requires Full Preprocessing.
<https://wg21.link/p1703r1>
- [P1754R1] Herb Sutter, Casey Carter, Gabriel Dos Reis, Eric Niebler, Bjarne Stroustrup, Andrew Sutton, Ville Voutilainen. 2019. Rename concepts to `standard_case` for C++20, while we still can.
<https://wg21.link/p1754r1>
- [P1766R1] Richard Smith. 2019. Mitigating minor modules maladies.
<https://wg21.link/p1766r1>
- [P1771R1] Peter Sommerlad. 2019. `[[nodiscard]]` for constructors.
<https://wg21.link/p1771r1>
- [P1811R0] Richard Smith, Gabriel Dos Reis. 2019. Relaxing redefinition restrictions for re-exportation robustness.
<https://wg21.link/p1811r0>
- [P1814R0] Mike Spertus. 2019. Wording for Class Template Argument Deduction for Alias Templates.
<https://wg21.link/p1814r0>
- [P1816R0] Timur Doumler. 2019. Wording for class template argument deduction for aggregates.
<https://wg21.link/p1816r0>
- [P1823R0] Nicolai Josuttis, Ville Voutilainen, Roger Orr, Daveed Vandevoorde, John Spicer, Christopher Di Bella. 2019. Remove Contracts from C++20.
<https://wg21.link/p1823r0>
- [P1825R0] David Stone. 2019. Merged wording for P0527R1 and P1155R3.
<https://wg21.link/p1825r0>
- [P1902R0] Barry Revzin. 2019. Missing feature-test macros 2018-2019.
<https://wg21.link/p1902r0>
- [SD6] SG10 Feature Test Recommendations.
<https://wg21.link/sd6>